ESGI - AL5 / T1

Apprentissage par renforcement

TP MDP : Snake

Processus de décision markoviens et Q-Learning

Rapport - Exercice 1

RUSESCU Daniel Alexandru
BAH Sulaiman
SEDJAI Fethi

Enseignant : Sylvain Gault
Date : 5 novembre 2026

Répartition du travail

Toutes les parties ont été réalisées ensemble par les trois membres du groupe : conception, code, tests, graphiques, analyse, rédaction et relecture. La mention « Travail collectif » associée à chaque question renvoie à cette répartition commune.

Code source : dépôt GitHub du groupe — github.com/Sulaimanzy/mdp-snake-alex-sulaiman-fethi

Sommaire

Reproduction des résultats

Exercice 1 — Jeu de base

- Question 1.1 — Classe ShortSnake et grille
- Question 1.2 — Position et direction initiales
- Question 1.3 — Position aléatoire du fruit
- Question 1.4 — Score
- Question 1.5 — Méthode move
- Question 1.6 — Franchissement d'un bord
- Question 1.7 — Fruit mangé
- Question 1.8 — Affichage __str__

Reproduction des résultats

- Python 3.14 (toute version ≥ 3.8 convient). Le jeu n'a besoin d'aucune dépendance ; `markdown`, `pygments` et `pymupdf` ne servent qu'à générer ce rapport.
- Depuis la racine du dépôt :

```
py -m pip install -r requirements.txt   # dépendances (rapport)
py -m unittest discover tests           # tests de la mécanique du jeu
py ex1_demo.py                          # partie scriptée de démonstration
py ex1_demo.py --play                   # jouer au clavier (z q s d + Entrée)
py rapport/build_pdf.py                 # régénère ce rapport en PDF
```

Fichiers :

Fichier	Contenu
`snake.py`	classe `ShortSnake` (mécanique du jeu)
`ex1_demo.py`	démonstration de l'exercice 1
`tests/test_snake.py`	tests unitaires de `ShortSnake`
`rapport/rapport.md`	source de ce rapport
`rapport/build_pdf.py`	conversion du rapport en PDF (Markdown → HTML → PDF via Edge ou Chrome)

Exercice 1 — Jeu de base

Convention : les coordonnées sont des tuples `(ligne, colonne)`, `(0, 0)` étant le coin en haut à gauche. Les directions sont les chaînes `UP`, `DOWN`, `LEFT`, `RIGHT`.

```
EMPTY, SNAKE, FRUIT = 0, 1, 2

UP, DOWN, LEFT, RIGHT = "UP", "DOWN", "LEFT", "RIGHT"
DIRECTIONS = {UP: (-1, 0), DOWN: (1, 0), LEFT: (0, -1), RIGHT: (0, 1)}
OPPOSITE = {UP: DOWN, DOWN: UP, LEFT: RIGHT, RIGHT: LEFT}

WALL_PENALTY = -10
FRUIT_REWARD = 10
```

Question 1.1 — Classe `ShortSnake` et grille

Travail collectif

```
class ShortSnake:
    def __init__(self, size=10, seed=None):
        if isinstance(size, int):
            size = (size, size)
        self.n_rows, self.n_cols = size
        if self.n_rows < 1 or self.n_cols < 1 or self.n_rows * self.n_cols < 2:
            raise ValueError("la grille doit contenir au moins 2 cases")
        self.grid = [[EMPTY] * self.n_cols for _ in range(self.n_rows)]
        self.rng = random.Random(seed)
```

La taille est un entier (grille carrée) ou un couple `(lignes, colonnes)`. La grille est une liste de listes dont chaque case vaut `EMPTY`, `SNAKE` ou `FRUIT`. Il faut au moins 2 cases pour pouvoir placer le serpent et le fruit. Le générateur `rng` peut recevoir une graine pour rendre une partie reproductible.

Question 1.2 — Position et direction initiales

Travail collectif

```
        self.snake_pos = (self.n_rows // 2, self.n_cols // 2)
        self.snake_dir = RIGHT
        self._set_cell(self.snake_pos, SNAKE)

    def _set_cell(self, pos, value):
        row, col = pos
        self.grid[row][col] = value
```

Le serpent démarre au milieu de la grille (`(5, 5)` pour une grille 10 × 10) et part vers la droite.

Question 1.3 — Position aléatoire du fruit

Travail collectif

```
        self.fruit_pos = self._place_fruit()

    def _place_fruit(self):
        free_cells = [
            (row, col)
            for row in range(self.n_rows)
            for col in range(self.n_cols)
            if self.grid[row][col] == EMPTY
        ]
        pos = self.rng.choice(free_cells)
        self._set_cell(pos, FRUIT)
        return pos
```

Le fruit est tiré uniformément parmi les cases libres, il n'apparaît donc jamais sous le serpent.

Question 1.4 — Score

Travail collectif

```
        self.score = 0
```

Question 1.5 — Méthode `move`

Travail collectif

```
    def move(self, direction=None):
        if direction is not None:
            if direction not in DIRECTIONS:
                raise ValueError(f"direction inconnue : {direction!r}")
            if direction != OPPOSITE[self.snake_dir]:
                self.snake_dir = direction

        d_row, d_col = DIRECTIONS[self.snake_dir]
        row, col = self.snake_pos[0] + d_row, self.snake_pos[1] + d_col
        reward = 0
        ...
        self._set_cell(self.snake_pos, EMPTY)
        self.snake_pos = (row, col)
        self._set_cell(self.snake_pos, SNAKE)
        ...
        self.score += reward
        return reward
```

`None` garde la direction courante. Une demande de demi-tour est ignorée : le serpent continue dans sa direction actuelle. La méthode renvoie la récompense de l'étape, qui servira au Q-Learning (exercice 3).

Question 1.6 — Franchissement d'un bord

Travail collectif

```
        if not (0 <= row < self.n_rows and 0 <= col < self.n_cols):
            row %= self.n_rows
            col %= self.n_cols
            reward += WALL_PENALTY
```

Le modulo fait réapparaître le serpent du côté opposé (ligne `-1` → ligne `n_rows - 1`, colonne `n_cols` → colonne `0`), avec une pénalité de 10 points.

Question 1.7 — Fruit mangé

Travail collectif

```
        if self.snake_pos == self.fruit_pos:
            reward += FRUIT_REWARD
            self.fruit_pos = self._place_fruit()
```

Le test a lieu après le déplacement de la tête : la case du fruit est déjà marquée `SNAKE`, donc le nouveau fruit est tiré parmi les autres cases. Si le serpent franchit un bord et tombe sur le fruit, les deux récompenses s'additionnent (−10 + 10 = 0).

Question 1.8 — Affichage `__str__`

Travail collectif

```
HEAD_SYMBOL = {UP: "^", DOWN: "v", LEFT: "<", RIGHT: ">"}
CELL_SYMBOL = {EMPTY: ".", FRUIT: "*"}

    def __str__(self):
        border = "+" + "-" * self.n_cols + "+"
        lines = [border]
        for row in self.grid:
            cells = (
                HEAD_SYMBOL[self.snake_dir] if cell == SNAKE else CELL_SYMBOL[cell]
                for cell in row
            )
            lines.append("|" + "".join(cells) + "|")
        lines.append(border)
        lines.append(f"Score : {self.score}")
        return "\n".join(lines)
```

La tête est dessinée par une flèche qui indique sa direction. Extrait de `py ex1_demo.py` (grille 5 × 8, graine 42) :

```
Étape 5 : move(UP) -> récompense +10 (on mange le fruit : +10)
+--------+
|.*.....^|
|........|
|........|
|........|
|........|
+--------+
Score : 10

Étape 6 : move(UP) -> récompense -10 (bord du haut franchi : on réapparaît en bas, -10)
+--------+
|.*......|
|........|
|........|
|........|
|.......^|
+--------+
Score : 0
```

Les 17 tests de `tests/test_snake.py` vérifient chacun de ces comportements (position initiale, demi-tour ignoré, passage des quatre bords, fruit mangé, cohérence de la grille, affichage).